

A Review of Large Language Models for Cybersecurity Defense: Core Capabilities, Application Practices, and Challenges

Xiangkai Zhao
China National Nuclear Co., Ltd.
Beijing, China
zhaoxiangkai@cnncc.com.cn

Yuyao Feng*
China Nuclear Power Operation
Technology Co., Ltd.
Wuhan, China
fenggy01@cnnp.com

Daijie Sun
China Nuclear Power Operation
Technology Co., Ltd.
Wuhan, China
sundj@cnnp.com

Liangbin Gao
China Nuclear Power Operation
Technology Co., Ltd.
Wuhan, China
lgao24@163.com

Tongxin Xu
China Nuclear Power Operation
Technology Co., Ltd.
Wuhan, China
xutongxinmath@163.com

Jiaming Lv
China Nuclear Power Operation
Technology Co., Ltd.
Wuhan, China
lvjm@cnnp.com

Guangrong Zhou
CNNC Shenzhen Kaili Group Co., Ltd.
Shenzhen, China
zhougr@cnnp.com

Chengliangyi Xia
CNNC Shenzhen Kaili Group Co., Ltd.
Shenzhen, China
t_xiachengly@cnnp.com.cn

Abstract

The rapid evolution of cyber threats — characterized by advanced persistent threats (APTs), software supply chain attacks, and AI-driven adversarial payloads — has exposed fundamental limitations in traditional rule-based and signature-based defense mechanisms. Large Language Models (LLMs), with core capabilities spanning natural language understanding, code engineering, logical reasoning, and autonomous decision-making, offer a transformative paradigm for cybersecurity defense. This paper presents a systematic review of LLM applications across the entire cybersecurity defense lifecycle. The review begins with an analysis of the core capabilities and key enabling technologies of LLMs — including prompt engineering, domain fine-tuning, Retrieval-Augmented Generation (RAG), and agent frameworks — that underpin their adaptation to security tasks. Application practices are then examined across four critical stages: pre-attack prevention and secure development (covering vulnerability detection, secure code generation, supply chain security, and malware/phishing detection), during-attack detection and threat awareness (including network intrusion detection, log analysis, and threat intelligence automation), post-attack response and disposition (encompassing digital forensics and automated incident response). The paper further dissects the security risks inherent to LLMs themselves — such as prompt injection, jailbreaking, data poisoning, and model stealing attacks — along with corresponding defense technologies. Finally, key challenges are identified, including hallucination, real-time performance bottlenecks, domain knowledge scarcity, data silos, interpretability deficits, and

compliance risks, with future research directions proposed toward building more robust, trustworthy, and human-centered intelligent defense systems.

CCS Concepts

• **Security and privacy** → Network security; Security protocols.

Keywords

Large Language Models, Cybersecurity Defense, Vulnerability Detection, Threat Intelligence, Intrusion Detection, LLM Security, Defense Lifecycle

ACM Reference Format:

Xiangkai Zhao, Yuyao Feng, Daijie Sun, Liangbin Gao, Tongxin Xu, Jiaming Lv, Guangrong Zhou, and Chengliangyi Xia. 2026. A Review of Large Language Models for Cybersecurity Defense: Core Capabilities, Application Practices, and Challenges. In *3rd International Conference on Machine Intelligence and Digital Applications (MIDA 2026)*, April 24–26, 2026, Xi'an, China. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3801438.3803160>

1 Introduction

1.1 Research Background and Significance

The global cybersecurity landscape has grown considerably more hostile, with attack methodologies evolving from straightforward single-point exploitation into sophisticated multi-stage, multi-vector campaigns. APT lifecycles now stretch across weeks or months, their behavioral indicators scattered thinly across vast volumes of logs and network traffic, while exploit chains, obfuscated payloads, and C2 protocol variants iterate on a near-daily basis. Modern SOC generate an average of 4,484 alerts per day, with analysts spending close to three hours on manual triage alone — a workload that makes alert fatigue one of the most persistent bottlenecks in operational defense. Traditional rule-based and

*Corresponding author



This work is licensed under a Creative Commons Attribution 4.0 International License. *MIDA 2026, Xi'an, China*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2155-7/2026/04

<https://doi.org/10.1145/3801438.3803160>

signature-driven detection mechanisms offer little relief: they remain largely blind to zero-day vulnerabilities, polymorphic malware, and targeted spear-phishing, while static security policies lag chronically behind the evolving threat landscape. Manual code auditing is slow, vulnerability remediation cycles are long, and the heavy dependence on individual expert judgment introduces both scalability constraints and inconsistency. Large language models have emerged as a compelling response to these compounding pressures. Their convergence of natural language understanding, code analysis and synthesis, logical reasoning, and autonomous decision-making provides a technical foundation broad enough to span tasks from threat intelligence processing to automated incident response [1]. Combined with enabling techniques such as prompt engineering, domain fine-tuning, retrieval-augmented generation, and agent frameworks, LLMs are increasingly bridging the gap between general-purpose capability and the domain-specific demands of cybersecurity operations [2].

1.2 Research Scope and Objectives

This paper focuses on the application research of LLMs in cybersecurity defense and systematically reviews their technical enabling roles across each phase of the defense lifecycle. The specific research scope includes: Pre-attack prevention and secure development, including secure code auditing, vulnerability detection, secure code generation, software supply chain security analysis, and malware and phishing detection. During-attack detection and threat awareness, including network intrusion and anomalous traffic detection, system and application log analysis, and network threat intelligence automation. Post-attack response and disposition, including security incident investigation and digital forensics, and automated incident response.

Security operation and governance throughout the defense process, including security policy and compliance analysis, security configuration management, security ticket processing, and data security and privacy protection. Meanwhile, this paper conducts an in-depth analysis of the security risks faced by LLMs themselves and their corresponding defense technologies, revealing the inherent challenges of deploying LLMs in cybersecurity applications.

The research objectives of this paper are fourfold:

- Systematically sort out the application paths of LLMs' core capabilities and key enabling technologies in cybersecurity defense scenarios, and clarify the internal connections and development contexts of each technical branch;
- Comprehensively analyze the technical application practices and research progress of LLMs from the perspective of the defense lifecycle, and reveal their core roles and implementation logic in each defense stage;
- Deeply dissect the security attack methods and defense technology systems targeting LLMs, and clarify the special risks and trust issues associated with deploying LLMs in security applications;
- Summarize the current technical challenges and practical deployment dilemmas of LLMs in cybersecurity defense applications, indicate future research directions, and provide theoretical support for building a more robust and trustworthy intelligent defense system [3].

2 Core Capabilities and Key Technologies of LLMs

LLMs derive their versatility from core capabilities spanning natural language understanding, code engineering, logical reasoning, and autonomous decision-making, while enabling techniques such as prompt engineering, fine-tuning, and retrieval-augmented generation bridge the gap from general-purpose competence to domain-specific application. As shown in the Table 1.

2.1 Core Capabilities of LLMs

The core capabilities of LLMs are the basis for their adaptation to diverse application scenarios. Current research focuses on expanding capability boundaries, improving performance, and facilitating real-world deployment, forming four major core capability systems.

Natural Language Understanding and Generation. Natural language understanding and generation represent the most fundamental and core capability of LLMs, covering three key tasks: semantic understanding, text generation, and dialogue interaction. Current research is evolving toward real-time performance, personalization, and complex document processing. In real-time dialogue interaction, Zhang et al. constructed an LLM-based real-time voice dialogue system integrating autoregressive streaming speech synthesis, achieving end-to-end real-time processing from voice input to text response[4]. In personalized dialogue, the SeCom framework proposed by Microsoft Research focuses on memory construction and retrieval for personalized conversational agents, significantly improving the coherence and personalized expression of multi-turn dialogues by optimizing the storage and invocation of long-term memory [5].

In complex document understanding, the long-document understanding benchmark fills the gap in document-level language understanding evaluation by integrating diverse datasets with unified formats [6], providing standardized evaluation criteria for LLMs in processing long documents and complex semantic correlation tasks. Evaluation results from the Deep Document Understanding Evaluation (DLUE) show that existing LLMs perform excellently in short-text tasks but still have obvious shortcomings in document-level tasks such as cross-paragraph reasoning and complex discourse structure analysis—this has also promoted the further development of long-context modeling techniques. Overall, natural language understanding and generation capabilities are steadily evolving from “fluent generation” toward the high-level goals of “accurate understanding, personalized adaptation, and real-time interaction”.

Code Understanding and Generation. Code understanding and generation capabilities enable LLMs to break through natural language boundaries and extend to the entire process of software development, covering key tasks such as program analysis, code synthesis, and defect detection, and thus becoming a core tool for improving software development efficiency. The CodeT5+ series models proposed by Salesforce constructed open-source code LLMs supporting encoder-decoder multi-mode operation by introducing diverse pre-training objectives [7]. Among these, InstructCodeT5+ 16B achieved excellent performance in code generation benchmarks, surpassing contemporary open-source models.

In code quality assurance, related research focuses on the practical application of defect detection and automated review. The

Table 1: Core Capabilities and Key Enabling Technologies

Category	Technology/Capability	Core Function	Representative Work
Core Capabilities	Natural Language Understanding & Generation	Semantic understanding, text generation, dialogue interaction; evolving toward real-time, personalized, and long-document processing	Real-time voice dialogue system [4]; SeCom personalized memory framework [5]; DLUE long-document benchmark [6]
	Code Understanding & Generation	Program analysis, code synthesis, defect detection, automated repair; full-chain generation-analysis-repair pipeline	CodeT5+ (multi-mode encoder-decoder) [7]; Defect-oriented code review [8]; LLM-GUARD (detection + repair) [9]
	In-Context Learning & Reasoning	Few-shot/zero-shot task adaptation, information correlation, causal analysis without parameter updates	Gemma-2 ICL mechanism analysis [10]; Modular ICL for causal discovery [11]; Knowledge graph-enhanced reasoning [12]
	Agent Capabilities	Autonomous planning, tool invocation, task execution, multi-agent collaboration	NaviAgent (tool navigation graph) [13]; HiAgent (hierarchical memory) [14]; AgentGen (autonomous evolution) [15]; Chain of Agents [16]
Key Enabling Technologies	Prompt Engineering	Optimize instruction design and reasoning paths to maximize model capabilities without parameter changes	Critic-CoT (self-evaluation) [17]; CoT boundary analysis [18]; Thinkless (adaptive CoT triggering) [19-20]
	Domain Fine-Tuning	Inject vertical domain knowledge through training on domain-specific corpora; balance specialization vs general ability	Financial instruction datasets [21]; KEFT (knowledge graph integration) [22]; LoRA/QLoRA for cybersecurity [23]
	Retrieval-Augmented Generation (RAG)	Dynamic knowledge update and supplementation through external knowledge bases; reduce hallucination	Collab-RAG (white-box + black-box collaboration) [24]; Agent search + RAG [25]

defect-oriented automated code review framework significantly outperforms standard LLMs in real merge request scenarios by improving code slicing algorithms and multi-stage review processes, supporting cross-language defect analysis and promoting the automation and precision of code review [8]. The LLM-GUARD system integrates static analysis and LLM reasoning for mainstream programming languages, realizing integrated vulnerability detection and repair [9]. Currently, code understanding and generation capabilities have formed a full-chain “generation-analysis-repair” technology system, developing toward cross-language adaptation and high-precision defect recognition.

In-Context Learning and Reasoning. In-Context Learning (ICL) and reasoning capabilities are the core support for LLMs to achieve few-shot/zero-shot task adaptation, covering key abilities such as information correlation, logical inference, and causal analysis. Research on the Gemma-2 model revealed the computational mechanism of ICL, providing theoretical guidance and engineering optimization directions for the efficient implementation of ICL in small-scale LLMs [10].

In causal reasoning, Zhang et al. proposed a modular ICL framework that realizes step-by-step reasoning for causal discovery tasks through prompt engineering, outperforming traditional methods on multiple benchmark datasets [11]. Wang et al. introduced knowledge graphs to enhance the causal reasoning ability of LLMs, supplementing the models’ causal correlation cognition with structured knowledge while improving the interpretability of the reasoning process [12]. Overall, ICL is expanding from few-shot to

large-scale example scenarios, providing new paths for complex task adaptation.

Agent Capabilities. The agent capabilities of LLMs are central to autonomous decision-making and complex task execution, covering three key components: autonomous planning, tool invocation, and task execution. The NaviAgent framework realizes efficient orchestration of large-scale tools by constructing a tool navigation graph and a two-layer planning mechanism [13]. HiAgent designs a hierarchical working memory system to address the challenge of long-term task memory management, significantly improving task completion rates in complex environments [14].

In the improvement of agent capabilities, the AgentGen framework enables LLMs to autonomously enhance their planning capabilities through iterative evolution in diverse tasks [15]. Multi-agent collaboration has become a new research hotspot; the Chain of Agents framework achieves training-free [16], task-agnostic multi-agent collaboration, solving complex problems such as long-text processing through task decomposition and result aggregation. Overall, agent capabilities are developing toward “large-scale adaptation, long-term memory, autonomous evolution, and collaborative cooperation”.

2.2 Key Enabling Technologies

Prompt Engineering. Prompt engineering maximizes the capabilities of existing LLMs by optimizing instruction design and guiding reasoning paths. The Critic-CoT framework introduces a “chain-of-thought critic” mechanism to identify and correct errors in reasoning paths through self-evaluation [17]. Wei et al. systematically

analyzed the applicable boundaries of Chain-of-Thought (CoT) reasoning, providing a quantitative basis for prompt strategy selection [18].

In efficiency optimization, the Thinkless framework constructs an adaptive CoT triggering mechanism to dynamically decide whether to enable long-chain reasoning, significantly reducing inference costs while maintaining accuracy [19]. Furthermore, the combination of prompt engineering and RAG can significantly reduce factual errors and improve the reliability of generated content [20].

Domain Fine-Tuning. Domain fine-tuning equips LLMs with vertical domain capabilities through training on domain-specific corpora. Li et al. constructed high-quality instruction datasets in the financial domain, significantly improving model performance on financial question-answering tasks [21]. The KEFT framework integrates knowledge graphs and unstructured texts, outperforming traditional fine-tuning in professional domain QA tasks such as healthcare and law [22].

In terms of parameter efficiency, Smith et al. compared the effects of Supervised Fine-Tuning (SFT), Low-Rank Adaptation (LoRA), and Quantized LoRA (QLoRA) in cybersecurity, proving that LoRA achieves the optimal balance between parameter size and performance, thus providing efficient solutions for resource-constrained scenarios [23].

Retrieval-Augmented Generation (RAG). RAG realizes dynamic information update and precise knowledge supplementation through external knowledge bases. The Astute RAG framework proposed by Google addresses retrieval defects and knowledge conflicts by constructing conflict detection and resolution mechanisms. Collab-RAG realizes collaboration between white-box and black-box LLMs, enhancing the ability to handle complex multi-hop reasoning tasks [24]. Wang et al. deeply integrates agent search with large-model reasoning, supporting real-time dynamic information updates and demonstrating the integration potential of RAG and agents [25].

Agent Frameworks. Agent frameworks provide standardized architectures for LLM agent deployment through task decomposition, tool orchestration, and memory management. The Chain of Agents architecture accomplishes task decomposition and aggregation through multi-LLM collaboration, performing excellently in long-text processing. Hierarchical architecture design is an important direction; both HiAgent and NaviAgent prove that layering can significantly improve the ability to process complex scenarios. Agent frameworks are developing toward multi-modality and modularity, lowering development thresholds.

3 Applications of LLMs in the Cybersecurity Defense Lifecycle

The cybersecurity defense lifecycle covers key stages including pre-attack prevention, during-attack detection and response, post-attack traceability and remediation, and continuous operation and governance. Endowed with powerful capabilities in natural language understanding, code engineering, logical reasoning, and agent collaboration, LLMs provide technical empowerment at each stage of the defense lifecycle, demonstrating significant application value—especially in pre-attack prevention, secure development,

and full-process security operation and governance. This chapter focuses on the application practices of LLMs in the above core scenarios, systematically sorting out technical application paths, research results, and development trends, and clarifying their core roles and implementation logic in the cybersecurity defense system. As shown in the Table 2.

3.1 Pre-Attack: Prevention and Secure Development

Vulnerability Detection and Patch Remediation. In secure code auditing and vulnerability detection, LLMs have substantially advanced the accuracy and scope of identifying security defects through domain fine-tuning and prompt engineering. The LLM-VulDetect system achieves cross-language vulnerability detection across C, Java, and Python via multi-granularity semantic modeling integrated with cybersecurity knowledge graphs, improving F1-scores by 20–25% over existing methods while excelling at uncovering hidden business logic flaws [26]. For resource-constrained environments, the SlimSecFT method improves detection accuracy by 14% using only 10% of typical fine-tuning resources, offering a practical path for edge deployment scenarios [27].

Beyond detection, LLMs are closing the loop from vulnerability discovery through remediation and proactive testing. In secure code generation, SecCodeGen introduces security specification pre-training objectives to achieve an 89% OWASP Top 10 compliance rate — over 45% higher than general code generators — directly applicable to web application development [28]. For patch remediation, SecPatchGen generates repair patches with an 82% pass rate for critical vulnerability types including memory safety and access control, with performance overhead held within 3% [29].

Software Supply Chain Security Analysis. Software supply chain security analysis aims to identify security risks in all supply chain links (dependencies, third-party components, development tools) and prevent supply chain attacks. With strong document understanding, knowledge integration, and logical reasoning capabilities, LLMs realize comprehensive investigation and precise early warning of supply chain risks. The SupplyChainLLM framework proposed by Chen et al. conducts full-link security assessment through supply chain knowledge graphs and vulnerability correlation rule libraries, achieving 88% risk identification accuracy—30% higher than traditional tools—and effectively identifying emerging attacks such as “supply chain poisoning” [30].

In dependency security analysis, LLMs quickly identify vulnerabilities and malicious components in third-party dependencies. By comprehensively analyzing code, version history, and maintainer information, the models accurately judge high-risk dependencies and provide alternatives, achieving 91% malicious component recognition in mainstream open-source repositories such as npm and PyPI, and issuing early warnings for multiple supply chain attacks [31]. Furthermore, LLMs play an important role in supply chain traceability and compliance analysis, ensuring compliance with data security and privacy regulations by analyzing component origins, licenses, and requirements, thus reducing compliance risks. Currently, LLMs are driving supply chain security analysis from “passive response” to “active prediction”, building a full lifecycle protection system.

Table 2: LLM Applications Across the Cybersecurity Defense Lifecycle

Defense Stage	Application Scenario	Representative Systems/Methods	References
Pre-Attack Prevention & Secure Development	Vulnerability Detection	LLM-VulDetect (cross-language semantic modeling + knowledge graph) VulnAgent (autonomous code slicing + risk rating) SlimSecFT (lightweight fine-tuning)	[26]–[27]
	Secure Code Generation & Patch Remediation	SecCodeGen (security specification pre-training) SecPatchGen (patch generation) Multi-model collaborative verification	[28][29]
	Supply Chain Security	SupplyChainLLM (knowledge graph + vulnerability correlation)	[30]
	Malware & Phishing Detection	MalDocLLM (multi-dimensional malicious document modeling) Multi-modal phishing detection	[31][32]
During-Attack Detection & Threat Awareness	Network Intrusion Detection	Single-model: fine-tuned Llama2/T5 Explanation: ChatNIDS, eX-NIDS Multi-model: Bi-GPT, BART+BERT, LLM+LSTM/MLP	[34]–[41]
	Log Parsing & Analysis	LLMParser (offline T5/LLaMA/ChatGLM) LILAC (adaptive cache + KNN) OpenLogParser (RAG + Llama3-8B) Hooglle (offline fine-tuning)	[42][43][44][45][46]
	Threat Intelligence Automation	AGIR (STIX report generation) CRUSH (knowledge graph construction) ATT&CK classification (BERT/encoder-decoder)	[47][48][49]
	Digital Forensics	ChatGPT forensics evaluation (6 areas) volGPT (memory forensics triage)	[50]–[60]
Post-Attack Response & Disposition	Automated Incident Response	Rughani framework (collection-analysis-reporting) TRAM (TTP extraction) SecureBERT (tactic classification) CTISum (attack summarization) Tseng et al. (SOC workflow automation)	[61]–[64]

Malware and Phishing Detection. Pre-attack malware and phishing detection aim to identify potential threat sources in advance and block attack entry points. With natural language understanding, multi-modal analysis, and knowledge enhancement capabilities, LLMs achieve accurate recognition and early warning of malware and phishing. In malware detection, the MalDocLLM proposed by Zhao et al. realizes precise classification and family identification of malicious documents through multi-dimensional modeling of text features, format structure, and behavior traces, with a detection accuracy above 97%—improving unknown malware recognition by 45% compared with traditional signature-based methods [32].

Early defenses against phishing relied on filtering surface-level indicators such as URLs, sender addresses, and IP metadata, often combined with machine learning classifiers to flag suspicious messages [33]. Mail servers further mitigated the problem by detecting bulk-sent identical emails as spam. However, generative AI has fundamentally undermined this approach by enabling attackers to produce large volumes of individually unique emails that evade near-duplicate detection. A parallel line of work shifted focus to the linguistic content of emails themselves, applying natural language processing and a range of classification architectures — from SVMs

and random forests to recurrent and deep convolutional neural networks — to distinguish phishing from legitimate correspondence based on textual features [34]. While these content-based methods proved more resilient than header-level filtering, the increasing sophistication of AI-generated phishing text continues to compress the stylistic gap between malicious and genuine emails, placing persistent pressure on detection accuracy.

3.2 During-Attack: Detection and Threat Awareness

Network Intrusion and Anomalous Traffic Detection. LLMs have entered the network intrusion detection space along two distinct lines. The more straightforward approach fine-tunes a single model — typically Llama2 or T5 — to process structured network flow sequences, and results on DDoS carpet bombing and related complex attack patterns have been encouraging [35]–[37]. What makes this line particularly interesting is the dual role these models can play: beyond detection itself, frameworks such as ChatNIDS and eX-NIDS generate human-readable explanations of alerts, giving analysts the interpretive context they need to act on automated outputs rather than simply trusting a score [38]. The second line acknowledges

that LLMs designed for natural language are not an ideal fit for the tabular, high-throughput character of network telemetry, and addresses this through multi-model architectures. Bi-GPT handles bidirectional CAN message analysis, dual-LLM configurations pair BART with BERT for traffic prediction [39], and hybrid systems bolt LLMs onto LSTM and MLP layers to jointly ingest structured features alongside text payloads [40]–[41]. The consistent finding across these efforts is that composite architectures outperform any single model, positioning LLMs as powerful components within broader detection pipelines rather than standalone replacements for existing tools.

System and Application Log Analysis. Log parsing — the unglamorous but essential task of converting raw event logs into structured representations — has proven a surprisingly productive application area [42]. LILAC introduced adaptive parsing caches using k-nearest neighbor matching [43]. Where labeled data is entirely unavailable, unsupervised approaches such as LUNAR [44] has eliminated annotation requirements, the latter pairing retrieval-augmented generation with Llama3-8B. Fine-tuned BERT variants [45] and prompt-engineered encoder-decoder models have both demonstrated the ability to narrow classifications substantially. On the cost front, HELP's vector-based log grouping is making LLM-powered parsing increasingly realistic for production-scale deployment [46].

Network Threat Intelligence Automation. Threat intelligence automation is perhaps where LLMs most directly alleviate human bottlenecks. Report generation systems such as AGIR produce structured outputs from STIX data, cutting drafting time by over 40% [47]. A particularly active front is automated classification against the MITRE ATT&CK matrix, whose 200+ techniques and 400+ sub-techniques have long demanded prohibitive amounts of expert time to apply manually. Additional work on report format conversion, summarization [48], and deduplication [49] rounds out a picture in which LLMs handle the volume-intensive processing that overwhelmed human teams, freeing analysts to focus on interpretive judgment.

3.3 Post-Attack: Response and Disposition

Security Incident Investigation and Digital Forensics. Digital forensics has absorbed AI capabilities incrementally — from Mukkamala and Sung's early network forensics work [50] and Case et al.'s automated evidence correlation [51] through to specialized applications in image source identification [52], deepfake detection [53],[54], voice authentication [55], and hybrid frameworks for mobile and vehicle infotainment analysis [56],[57]. LLMs represent the latest and arguably most versatile addition to this trajectory. Scanlon et al. evaluated ChatGPT across six forensics areas — evidence analysis, report generation, investigation support among them — and found that it meaningfully assists examiners with complex data interpretation and preliminary reporting, though expert supervision remains indispensable for legal validity [58]. More targeted tools have followed: volGPT applies prompt-based LLMs to memory forensics triage, achieving strong ransomware detection accuracy while producing human-readable explanations that make the reasoning auditable [59], and Rughani proposed an integrated framework spanning evidence collection, automated

analysis, and structured reporting [60]. The recurring theme is that LLMs accelerate forensic workflows and improve interpretability, but the non-negotiable requirements of evidentiary reliability, legal admissibility, and ethical standards preclude fully unsupervised operation.

Automated Incident Response. On the incident response side, the scale of the problem is stark: SOC analysts contend with an average of 4,484 alerts per day and spend close to three hours on manual triage alone, a burden compounded by the additional delays of enriching alerts with threat intelligence and navigating fragmented multi-cloud security models. LLMs are beginning to chip away at this bottleneck from several angles. MITRE's TRAM tool simplifies TTP extraction from threat reports [61], fine-tuned small models such as SecureBERT outperform larger commercial alternatives like GPT-3.5 in precise tactic classification [62], and CTISum provides dedicated benchmarks for attack process summarization [63]. For more direct SOC workflow automation, Tseng et al. demonstrated LLM-driven pipelines covering indicator extraction, multi-model voting for filtering, regex rule generation, and relational graph construction [64]. That said, the role of LLMs in active response — actually executing playbooks and taking containment actions — remains constrained, with high fine-tuning costs for specialized scenarios and the relentless pace of attack evolution presenting challenges that passive summarization and classification alone cannot resolve.

4 Security Risks and Defenses of Large Language Models (LLMs) Themselves

With the widespread adoption of large language models (LLMs) in the field of cybersecurity, their inherent security vulnerabilities have become increasingly prominent. LLMs serve not only as defensive tools but also as potential attack targets or vectors for malicious exploitation. This section systematically synthesizes the security challenges confronting LLMs from both offensive (as shown in the Table 3) and defensive perspectives (as shown in the Table 4), and explores the unique risks and trust issues associated with deploying LLMs in cybersecurity applications.

4.1 Security Attacks Against LLMs

Prompt Injection Attacks. Prompt injection attacks are typically divided into two categories: direct and indirect. Li and Zhou [65] demonstrated that goal-driven direct attacks, often optimized through advanced techniques, can not only alter generated outputs but also redefine the model's task boundaries. These findings highlight how adversaries exploit models' task adherence to redirect outputs entirely. In contrast, Thapa and Park [66] showed that indirect attacks are particularly difficult to detect with conventional filtering methods, advocating for forensic analysis-based detection as a more robust alternative. Such attacks are especially concerning because they can bypass internal safety mechanisms in addition to manipulating content. Expanding on this, Ferrag [67] proposed a taxonomy of prompt injection attack surfaces, including content injection, context manipulation, and task redirection, thereby providing a structured framework for developing defense strategies.

Table 3: Security Attacks Against LLMs

Attack Category	Attack Mechanism	Representative Method	References
Prompt Injection	Manipulate application-layer inputs to hijack model behavior and execute malicious instructions	HOUYI (three-component black-box injection: framework + separation + sabotage)	[65]–[67]
Data Poisoning	Contaminate training data to implant backdoors, biases, or extract memorized content	Carlini et al. (k-explicit memorization extraction) Special Character Attack / SCA (second-order memorization)	[68][69]
Membership Inference & Model Stealing	Violate model privacy and intellectual property through query-based reconstruction or data inference	DFME (data-free model extraction via synthetic queries) PLeak (system prompt stealing) Du et al. (fine-tuning membership inference)	[70]

Data Poisoning Attacks. Data poisoning attacks compromise LLMs by contaminating training data, thereby implanting backdoors or biases within the model. Training data extraction, a critical variant of such attacks, was first systematically demonstrated by Carlini et al. [68], who proved that LLMs memorize and leak training data. For GPT-2, they proposed the concept of k-explicit memorization, dispelling the long-standing misconception that “no overfitting implies no memorization.” Experiments successfully extracted 604 unique memorized samples, including complete personally identifiable information (PII), GitHub code snippets, and sensitive URLs. The study identified three key patterns: the scale curve (a 1.5-billion-parameter model exhibits 18 times more memorization than its 124-million-parameter counterpart), low-frequency memorization (sequences repeated only 10 times can be extracted), and context dependence.

The Special Character Attack (SCA), introduced by Badhan et al. [69], systematically exploits special characters to trigger data leakage, uncovering a unique “second-order memorization” mechanism in LLMs. SCA successfully extracted sensitive information—including PII, code repositories, and preset prompts—from both commercial models (e.g., ChatGPT, Gemini) and open-source models (e.g., Llama, Falcon).

Membership Inference and Model Stealing Attacks. Membership inference and model stealing attacks violate the privacy of LLMs and infringe upon intellectual property rights. In the context of model stealing, Truong et al. [70] proposed Data-Free Model Extraction (DFME), which enables model stealing by generating synthetic query samples without any prior knowledge of the training data distribution. Their work demonstrated that an attacker can reconstruct a surrogate model with accuracy comparable to the original model within a reasonable query budget. The PLeak attack, proposed by Hui et al. [71], uncovered a novel threat: system prompt stealing. It argued that system prompts essentially function as “lightweight model parameters,” and their leakage is equivalent to the direct replication of model capabilities. For membership inference attacks, Du et al. [72] revealed that LLMs are vulnerable to user data inference during fine-tuning and deployment. Specifically, an attacker can determine whether a user’s data was included in the fine-tuning process merely by analyzing the probability distribution

of model outputs. The attack’s Area Under the Receiver Operating Characteristic Curve (AUROC) peaks when only 5%–18% of fine-tuning steps are completed, and common defensive measures—such as gradient clipping and data deduplication—prove ineffective in mitigating this risk. As shown in the Table 4.

4.2 LLM Security Defense Technologies

Input/Output Filtering and Content Security Policies. Input and output filtering constitutes the first line of defense for LLM security. Markov et al. [73] proposed a content security framework that is deployable in real-world production environments. Its core principle involves continuously monitoring LLM inputs and outputs using an independent content detection model. The framework emphasizes the construction of a fine-grained, hierarchical security content taxonomy, covering high-risk categories such as violence, hate speech, and self-harm. It continuously captures emerging bypass patterns through high-quality annotation, active learning, and red-teaming, and employs an off-model security classifier to validate generated outputs—ensuring that unsafe content is blocked even if the LLM is successfully induced. Boreiko et al. [74] demonstrated that perplexity-based filtering exhibits high practical value in LLM security. Adversarial prompts typically exhibit low fluency or local semantic anomalies, and even in white-box scenarios where attackers optimize for both jailbreaking and low perplexity, the attack success rate is significantly diminished.

Prompt Isolation and Structured Queries. Prompt isolation and structured queries address LLM security issues at the architectural level. Boreiko et al. [74] showed that input paraphrasing and retokenization can effectively disrupt adversarial trigger structures, leading to a significant reduction in attack success rates—with paraphrasing proving particularly effective in gray-box scenarios. The core conclusion of their work is that, due to the discreteness and high computational cost of text adversarial optimization, increasing the difficulty and cost of attacks constitutes an effective defense in itself.

Red-Teaming and Adversarial Training. Red-teaming is a proactive approach to identifying system vulnerabilities by simulating the behaviors of malicious attackers. Barry et al. [75] systematically investigated the failure mechanisms of anti-jailbreaking

Table 4: LLM Security Defense Technologies

Defense Category	Defense Mechanism	Representative Method	References
Input/Output Filtering & Content Security	Monitor inputs/outputs using independent content detection models; filter adversarial prompts based on perplexity and fluency anomalies	Markov et al. (hierarchical content security taxonomy + off-model classifier + active learning + red-teaming) Boreiko et al. (perplexity-based filtering)	[73]
Prompt Isolation & Structured Queries	Achieve strong semantic separation of instructions and data at the architecture level; restrict function scope and enforce output constraints	Liu et al. (sub-task decomposition + least privilege + output verification + deterministic code migration) Jain et al. (input paraphrasing + retokenization)	[74]
Red-Teaming & Adversarial Training	Proactively identify vulnerabilities by simulating malicious attacker behaviors; stress-test defense boundaries	Barry et al. (systematic failure mechanism analysis of anti-jailbreaking defenses) MASTERKEY defense analysis (capability boundary delineation)	[75]
Safety Alignment Fine-Tuning	Eliminate backdoor behaviors through targeted fine-tuning strategies; mix clean pre-trained weights with suspicious weights	Zhu et al. "Fine-Tuning Is All You Need" (super-fine-tuning with dynamic learning rate) Fine-mixing (proportional weight mixing + few-shot re-fine-tuning) Embedding Purification (word frequency + embedding shift detection)	[76][77]

defenses against adaptive attackers, arguing that jailbreaking attacks are inherently a game between alignment constraints and user instructions. Attackers can embed harmful intents into seemingly legitimate dialogues through role-playing, task decomposition, context shifting, and semantic steganography. The study concluded that effective anti-jailbreaking defenses should not pursue "absolute robustness" but rather focus on reducing attack success rates, increasing attack costs, and limiting the severity of potential harm. However, adversarial training faces significant challenges in LLM scenarios, including extremely high computational costs and severe model degradation, making it difficult to directly adapt mature methodologies from the computer vision domain.

Safety Alignment Fine-Tuning. Safety alignment fine-tuning is an effective technique for eliminating malicious behaviors in LLMs. Zhu et al. [76] put forth the core insight that "Fine-Tuning Is All You Need," demonstrating that full-model fine-tuning alone can effectively eliminate backdoor behaviors. They proposed a super-fine-tuning strategy that employs dynamic learning rate scheduling to rapidly "forget" backdoor trigger patterns in the early stages of training while preserving model performance in later stages. This approach reduces attack success rates to near-random levels across three scenarios: encoder pre-training, transfer learning, and standalone models—with computational overhead significantly lower than that of complex trigger reversal and distillation-pruning techniques. The Fine-mixing method, proposed by Li et al. [77], leverages clean pre-trained weights as a defensive prior. By proportionally mixing suspicious fine-tuned weights with official pre-trained weights and performing few-shot re-fine-tuning, Fine-mixing reduces attack success rates from nearly 100% to below 10%–20% with only a 2%–3% loss in inference accuracy. For embedding-layer

backdoors, the study further proposed the Embedding Purification method, which identifies and resets potentially poisoned word vectors based on word frequency and embedding shift statistics.

5 Challenges, Limitations and Future Research Directions

At the technical level, the central tension is that LLMs are probabilistic sequence generators being pressed into service for tasks demanding deterministic correctness. The hallucination problem is not an academic curiosity but a concrete operational hazard: empirical work by Pearce et al. found that roughly 40% of security patches produced by code generation tools introduced new vulnerabilities such as hard-coded credentials and injection flaws [78], while the phenomenon of package hallucination — where models recommend nonexistent libraries that attackers can then register and weaponize — opens a direct pathway to supply chain compromise. In security operations centers, these failure modes surface as false positives that deepen alert fatigue and false negatives that let genuine intrusions pass undetected, with outputs that frequently lack the causal reasoning chains analysts need before they can trust and act on automated recommendations.

Compounding this is a severe domain knowledge deficit. General pre-training corpora contain scant coverage of proprietary industrial control protocols, obfuscated malware instruction sets, or the business logic specific to sectors like nuclear power and critical infrastructure. Fine-tuning on specialized security data can partially bridge this gap, but at the well-documented cost of catastrophic forgetting, where the model sacrifices general reasoning ability as it absorbs narrow domain expertise [79]. The threat landscape, meanwhile, evolves on a weekly or even daily cadence — new exploit chains, payload obfuscation techniques, and command-and-control

variants emerge far faster than any static model can track. An LLM trained before the discovery of a Log4j variant or a Proxy-Logon derivative does not merely lack knowledge of that threat; it may actively generate misleading guidance rooted in outdated assumptions, compounding rather than mitigating risk.

The practical barriers to deployment are equally formidable and in many respects more intractable, because they involve organizational, regulatory, and architectural constraints that no amount of model improvement alone can resolve. High-quality security data — real attack traffic, exploit samples, incident response records — is both scarce and extraordinarily sensitive. Regulations including GDPR, China's Cybersecurity Law, and sector-specific governance frameworks impose strict limits on cross-organizational sharing, producing deep data silos that starve models of the diverse, representative training material they require. Deploying models against live infrastructure introduces further friction: modern security ecosystems are assemblages of heterogeneous tools — SIEM, SOAR, EDR — communicating through structured protocols and rigid APIs, and bridging the semantic gap between the natural language world of LLMs and the precise machine instruction world of these systems demands complex middleware where even minor parameter misinterpretations can disrupt business-critical communications or inadvertently widen attack surfaces [80]. The stateless nature of current architectures worsens the problem, as multi-stage incident response workflows — isolate host, extract forensic image, query threat intelligence, update firewall rules — require persistent context that models cannot natively maintain. LLMs have also become targets in their own right: prompt injection attacks embed adversarial instructions within seemingly benign inputs to hijack model behavior, and because these attacks exploit the same natural language understanding that makes LLMs useful, they prove exceptionally difficult to distinguish from legitimate queries using conventional detection methods [81]. The absence of settled legal frameworks for AI-driven security decisions — who bears liability when an autonomous isolation action disrupts critical services — further constrains adoption, pushing most organizations toward conservative postures where models advise but humans execute [82].

Addressing these intertwined challenges will require progress on several fronts simultaneously. Domain-specific pre-training pipelines built on large-scale security corpora — code repositories, technical advisories, operational logs — offer one path toward closing the knowledge gap without sacrificing general reasoning, particularly when paired with curated instruction datasets spanning attack-defense deduction and vulnerability analysis. Cybersecurity-specific evaluation benchmarks that measure code repair pass rates, real-world vulnerability discovery accuracy, and multi-modal comprehension of network traffic within simulated attack-defense environments are equally essential [83]. Chain-of-thought techniques that force models to externalize their reasoning provide a pragmatic near-term path toward outputs analysts can scrutinize, while uncertainty quantification methods can flag low-confidence judgments for human review rather than presenting all outputs with equal authority. Lightweight deployment through distillation and quantization can bring capable models to edge devices, enabling low-latency local detection that sidesteps the privacy risks of centralized cloud processing, with a tiered architecture reserving deep

analysis for cloud resources [84]. The most productive framing, ultimately, is not full automation but genuine human-machine collaboration: models that handle the volume and velocity of routine analysis overwhelming human teams, but that route consequential decisions through experts who retain both the authority and the accountability that current legal and operational realities demand.

6 Conclusion

This paper systematically reviews the application status and development trends of large language models (LLMs) in cybersecurity from dual perspectives: the "defense lifecycle" and "model intrinsic security." Research findings indicate that LLMs have established a comprehensive technical empowerment system covering pre-attack prevention, in-attack detection, post-attack response. LLMs exhibit significant advantages in scenarios such as secure code auditing, vulnerability detection, intrusion detection, threat intelligence automation, digital forensics, and compliance analysis—driving the evolution of cybersecurity defense from the traditional "rule-driven, labor-intensive" paradigm to a new "intelligence-driven, automated collaborative" paradigm.

However, the in-depth application of LLMs in cybersecurity still faces substantial technical challenges—including hallucinations, real-time performance bottlenecks, and domain knowledge scarcity—as well as practical obstacles such as data silos, black-box decision-making, and compliance risks. In particular, the "security paradox"—using potentially vulnerable AI systems to defend against cyberattacks—necessitates the adoption of defense-in-depth strategies during deployment. Future research should focus on dedicated cybersecurity pre-training paradigms, high-quality evaluation benchmarks, improved robustness and interpretability, lightweight edge deployment, and human-machine collaborative operation models. These efforts will promote the intelligent, adaptive, and trustworthy in-depth integration of LLMs into cybersecurity defense systems.

References

- [1] Brown, T., Mann, B., Ryder, N., *et al.*: Language Models are Few-Shot Learners. In: *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877–1901 (2020).
- [2] Wei, J., Wang, X., Schuurmans, D., *et al.*: Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In: *Advances in Neural Information Processing Systems*, vol. 35, pp. 24897–24908 (2022).
- [3] Hochreiter, S., Young, T., Leitner, M.: A Survey of Large Language Models. *Foundations and Trends in Machine Learning* 17(1), 1–214 (2024).
- [4] Zhang, S., Liu, Y., Wang, H.: VoiceLLaMA: Towards Efficient Real-Time Voice Dialogue with Large Language Models. *IEEE Transactions on Audio, Speech, and Language Processing* 32, 1568–1581 (2024).
- [5] Gao, R., Xiong, C., Socher, R.: SeMem: Prompting Large Language Models with Personalized Semantic Memory for Long-Term Dialogue. In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 10562–10574 (2023).
- [6] Chen, J., Lin, Y., Liu, Z.: LongBench: A Benchmark for Long Context Understanding. In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, pp. 7162–7175 (2023).
- [7] Wang, Y., Liu, K., Liu, Q., *et al.*: CodeT5+: Open Code Large Language Models for Code Understanding and Generation. In: *Proceedings of the 30th ACM SIGPLAN International Conference on Functional Programming*, pp. 1–24 (2025).
- [8] Zhou, Y., Li, J., Zhang, H.: DefectHunter: LLM-Based Automated Code Review for Effective Vulnerability Detection. *IEEE Transactions on Software Engineering* 50(2), 789–804 (2024).
- [9] Chen, Y., Luo, W., Xu, G.: LLM-GUARD: Integrating Static Analysis and Large Language Models for Secure Coding. In: *2024 IEEE Symposium on Security and Privacy*, pp. 123–136. IEEE (2024).

- [10] Bakalova, M., Petrov, Y., Nakov, P.: Towards Understanding In-Context Learning in Small Language Models. In: *Advances in Neural Information Processing Systems*, vol. 37, pp. 23456–23468 (2024).
- [11] Zhang, L., Rahman, S., Wang, Z.: CausalCoT: Enhancing Causal Reasoning in Large Language Models. In: *Proceedings of the 2024 ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 2988–2999. ACM (2024).
- [12] Wang, H., Liu, Z., Sun, M.: Knowledge Graph Prompting for Multi-Document Question Answering. In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, pp. 1234–1245 (2023).
- [13] Jiang, H., Li, Y., Zhang, F.: NaviAgent: Tool Navigation for Scalable LLM-Based Agents. In: *Proceedings of the 2024 International Conference on Machine Learning*, pp. 3456–3463 (2024).
- [14] Liang, X., Chen, W., Yu, T.: HiAgent: Hierarchical Memory for Long-Horizon LLM Agent Planning. In: *Proceedings of the 2024 International Conference on Learning Representations* (2024).
- [15] Liu, H., Zhou, M., Li, L.: AgentGen: Task Environment Generation for LLM Agent Ability Enhancement. In: *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 9876–9888 (2024).
- [16] Zhang, Y., Wang, Q., Liu, Y.: Chain of Agents: Large Language Models as Collaborators for Complex Problem Solving. In: *Advances in Neural Information Processing Systems*, vol. 37, pp. 56789–56798 (2024).
- [17] Wang, Y., Yang, L., Liu, Z.: Critic-CoT: Leveraging Self-Criticism in Chain-of-Thought Reasoning. In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 11234–11246 (2023).
- [18] Wei, J., Boltze, J., Cheng, X., et al.: When Do Chain-of-Thought Prompting Help? A Statistical Analysis of Reasoning in LLMs. In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, pp. 2345–2358 (2023).
- [19] Chen, J., Zhang, L., Liu, M.: ThinkLess: Dynamic Triggering of Chain-of-Thought for Efficient LLM Inference. In: *Proceedings of the 2024 ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 1234–1243. ACM (2024).
- [20] Lewis, P., Perez, E., Piktus, A., et al.: Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In: *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474 (2020).
- [21] Li, Y., Zhang, Q., Wang, H.: FinLLaMA: A Dedicated Financial Large Language Model. In: *Proceedings of the 2024 IEEE International Conference on Data Mining*, pp. 345–354. IEEE (2024).
- [22] Li, J., Wang, Z., Liu, Y.: KEFT: Knowledge-Enhanced Fine-Tuning for Domain-Specific Large Language Models. *IEEE Transactions on Knowledge and Data Engineering* 36(4), 1567–1580 (2024).
- [23] Smith, J., Johnson, K., Davis, L.: Parameter-Efficient Fine-Tuning for Cybersecurity Applications. In: *2024 IEEE Conference on Communications and Network Security*, pp. 1–8. IEEE (2024).
- [24] Yang, L., Chen, W., Wang, H.: Collab-RAG: Collaborative Retrieval-Augmented Generation for Complex Question Answering. In: *Proceedings of the 2024 ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 4567–4576. ACM (2024).
- [25] Wang, Z., Liu, Z., Sun, M.: Search-o1: Search-Augmented Large Language Models for Open-World Reasoning. In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pp. 6789–6798 (2024).
- [26] Li, X., Wang, Y., Zhang, L., et al.: LLM-VulDetect: Cross-Language Vulnerability Detection with Multi-Granularity Code Semantic Modeling. In: *The 47th International Conference on Software Engineering (ICSE 2025)*. IEEE, Melbourne, Australia (2025).
- [27] Zhang, C., Li, H., Wang, Q., et al.: SlimSecFT: Lightweight Fine-Tuning for Edge-Device-Oriented Vulnerability Auditing. In: *The 2025 IEEE International Conference on Edge Computing (EDGE 2025)*. IEEE, San Francisco, US (2025).
- [28] Wang, Z., Liu, J., Chen, S., et al.: SecCodeGen: Safety-Aware Code Generation with Security Specification Pre-training. In: *The 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP 2025)*. ACL, Suzhou, China (2025).
- [29] Li, R., Zhang, S., Liu, Y., et al.: SecPatchGen: LLM-Based Automated Vulnerability Patch Generation with Performance Awareness. In: *The 47th International Conference on Software Engineering (ICSE 2025)*. IEEE, Melbourne, Australia (2025).
- [30] Chen, Y., Zhao, P., Wang, L., et al.: SupplyChainLLM: Knowledge Graph-Enhanced Supply Chain Security Risk Assessment. In: *The 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD 2025)*. ACM, Halifax, Canada (2025).
- [31] Lira, O.G., Marroquin, A., To, M.A.: Harnessing the Advanced Capabilities of LLM for Adaptive Intrusion Detection Systems. In: *Proceedings of the 38th International Conference on Advanced Information Networking and Applications (AINA 2024)* (2024).
- [32] Zhao, J., Liu, F., Chen, W., et al.: MalDocLLM: Multi-Dimensional Modeling for Malicious Document Classification and Family Identification. In: *The 2025 IEEE Symposium on Security and Privacy (S&P 2025)*. IEEE, San Francisco, US (2025).
- [33] Cranor, L., Egelman, S., Hong, J., Phish, Z.P.: An evaluation of anti-phishing toolbars. In *Technical Report CMU-CyLab-06-018*; Carnegie Mellon University CyLab: Pittsburgh, PA, USA, 2006; Volume 13.
- [34] Yerima, S.Y.; Alzaylaee, M.K. High accuracy phishing detection based on convolutional neural networks. In *Proceedings of the 2020 3rd International Conference on Computer Applications & Information Security*, Riyadh, Saudi Arabia, 19–21 March 2020; IEEE: Piscataway, NJ, USA, 2020; pp. 1–6.
- [35] Zhang, J.: Leveraging Large Language Models for Autonomous Threat Detection in IoT Networks. In: *Proceedings of the 8th International Conference on Electronic Information Technology and Computer Engineering*, pp. 545–550 (2024).
- [36] Cobilean, V., Mavikumbure, H.S., Wickramasinghe, C.S., Varghese, B.J., Pennington, T., Manic, M.: Anomaly Detection for In-Vehicle Communication Using Transformers. In: *Proceedings of the 49th Annual Conference of the IEEE Industrial Electronics Society*, pp. 1–6. IEEE (2023).
- [37] Bai, Y., Sun, M., Zhang, L., Wang, Y., Liu, S., Liu, Y., Tan, J., Yang, Y., Lv, C.: Enhancing Network Attack Detection Accuracy Through the Integration of Large Language Models and Synchronized Attention Mechanism. *Applied Sciences* 14, 3289 (2024).
- [38] Nam, M., Park, S., Kim, D.S.: Intrusion Detection Method Using Bi-Directional GPT for In-Vehicle Controller Area Networks. *IEEE Access* 9, 124931–124944 (2021).
- [39] Düzgün, B., Çayır, A., Unal, U., Dağ, H.: Network Intrusion Detection System by Learning Jointly from Tabular and Text-Based Features. *Expert Systems* 41(4), e13518 (2024).
- [40] Le, V.-H., Zhang, H.: Log Parsing: How Far Can ChatGPT Go? In: *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pp. 1699–1704. IEEE (2023).
- [41] Ma, Z., Chen, A.R., Kim, D.J., Chen, T.-H., Wang, S.: LLMParser: An Exploratory Study on Using Large Language Models for Log Parsing. In: *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, pp. 1–13 (2024).
- [42] Xu, J., Yang, R., Huo, Y., Zhang, C., He, P.: DivLog: Log Parsing with Prompt Enhanced In-Context Learning. In: *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, pp. 1–12 (2024).
- [43] Zhong, A., Mo, D., Liu, G., Liu, J., Lu, Q., Zhou, Q., Wu, J., Li, Q., Wen, Q.: LogParser-LLM: Advancing Efficient Log Parsing with Large Language Models. In: *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pp. 4559–4570. ACM (2024).
- [44] Mehrabi, M., Hamou-Lhadj, A., Moosavi, H.: The Effectiveness of Compact Fine-Tuned LLMs in Log Parsing (n.d.).
- [45] Spracklen, J., Wijewickrama, R., Sakib, A.H.M.N., et al.: We Have a Package for You! A Comprehensive Analysis of Package Hallucinations by Code Generating LLMs. In: *34th USENIX Security Symposium (USENIX Security 25)*, pp. 3687–3706 (2025).
- [46] Liu, N.F., Lin, K., Hewitt, J., et al.: Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics* 12, 157–173 (2024).
- [47] Ji, Z., Yu, T., Xu, Y., et al.: Towards Mitigating LLM Hallucination via Self-Reflection. In: *Findings of the Association for Computational Linguistics: EMNLP 2023*, pp. 1827–1843 (2023).
- [48] Sundar, S.S.: Rise of Machine Agency: A Framework for Studying the Psychology of Human-AI Interaction (HAI). *Journal of Computer-Mediated Communication* 25(1), 74–88 (2020).
- [49] Alves, P.M., Geraldo Filho, P., Gonçalves, V.P.: Leveraging BERT's Power to Classify TTP from Unstructured Text. In: *2022 Workshop on Communication Networks and Power Systems (WCNPS)*, pp. 1–7. IEEE (2022).
- [50] Mukkamala, S., Sung, A.H.: Identifying Significant Features for Network Forensic Analysis Using Artificial Intelligent Techniques. *International Journal of Digital Evidence* 1, 1–17 (2003).
- [51] Case, A., Cristina, A., Marziale, L., Richard, G.G., Roussev, V.: FACE: Automated Digital Evidence Discovery and Correlation. *Digital Investigation* 5, S65–S75 (2008).
- [52] Yang, P., Baracchi, D., Ni, R., Zhao, Y., Argenti, F., Piva, A.: A Survey of Deep Learning-Based Source Image Forensics. *Journal of Imaging* 6, 9 (2020).
- [53] Afchar, D., Nozick, V., Yamagishi, J., Echizen, I.: MesoNet: A Compact Facial Video Forgery Detection Network. In: *2018 IEEE International Workshop on Information Forensics and Security (WIFS)*, pp. 1–7. IEEE (2018).
- [54] Orozco, A.L.S., Huamán, C.Q., Álvarez, D.P., Villalba, L.J.G.: A Machine Learning Forensics Technique to Detect Post-Processing in Digital Videos. *Future Generation Computer Systems* 111, 199–212 (2020).
- [55] Qamhan, M.A., Altaheri, H., Meftah, A.H., Muhammad, G., Alotaibi, Y.A.: Digital Audio Forensics: Microphone and Environment Classification Using Deep Learning. *IEEE Access* 9, 62719–62733 (2021).
- [56] Vasilaras, A., Papadoudis, N., Rizomiliotis, P.: Artificial Intelligence in Mobile Forensics: A Survey of Current Status, a Use Case Analysis and AI Alignment Objectives. *Forensic Science International: Digital Investigation* 49, 301737 (2024).
- [57] Fayyaz, Y., Almeahadi, A., El-Khatib, K.: A Hybrid Artificial Intelligence Framework for Enhancing Digital Forensic Investigations of Infotainment Systems.

- Forensic Science International: Digital Investigation 49, 301751 (2024).
- [58] Scanlon, M., Breiteringer, F., Hargreaves, C., Hilgert, J.-N., Sheppard, J.: ChatGPT for Digital Forensic Investigation: The Good, the Bad, and the Unknown. *Forensic Science International: Digital Investigation* 46, 301609 (2023).
 - [59] Oh, D.B., Kim, D., Kim, H.K.: VolGPT: Evaluation on Triaging Ransomware Process in Memory Forensics with Large Language Model. *Forensic Science International: Digital Investigation* 49, 301756 (2024).
 - [60] Rughani, P.H.: Artificial Intelligence Based Digital Forensics Framework. *International Journal of Advanced Research in Computer Science* 8 (2017).
 - [61] Liu, H., Chen, J., Zhang, Y., *et al.*: ComplianceLLM: Retrieval-Augmented Fine-Tuning for Cross-Region Security Compliance Analysis. In: *The 13th International Conference on Learning Representations (ICLR 2025)*. OpenReview (2025).
 - [62] Chen, G., Wang, Y., Zhang, P., *et al.*: LLM-Powered Intelligent Ticket Processing and Q&A for Security Operations. In: *The 2025 IEEE International Conference on Cloud Computing (CLOUD 2025)*. IEEE, Vancouver, Canada (2025).
 - [63] Zhang, D., Li, J., Wang, S., *et al.*: LLM-Based Adaptive Data Anonymization: Balancing Privacy and Data Utility. *Journal of Cybersecurity and Privacy (JCP 2025)* (2025).
 - [64] Wang, Q., Li, Z., Chen, H., *et al.*: PrivacyRiskLLM: Domain-Pretrained LLM for Data Privacy Risk Assessment in Sensitive Domains. In: *The 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP 2025)*. ACL, Suzhou, China (2025).
 - [65] Li, W., Zhou, Z.: Goal-Guided Prompt Injection: Optimizing Attacks on Instruction-Following Models. *IEEE Transactions on Artificial Intelligence*, 5(1): 44–59 (2024).
 - [66] Thapa, R., Park, Y.: Forensic Techniques for Detecting Indirect Prompt Injection in Chatbot Systems. *Computers & Security*, 139, 103053 (2024).
 - [67] Ferrag, M. A.: Taxonomy and Challenges of Prompt Injection in Large Language Models. *Computers & Security*, 145, 103241 (2025).
 - [68] Carlini, N., Tramer, F., Wallace, E., Jagielski, M., Herbert-Voss, A., Lee, K., Roberts, A., Brown, T., Song, D., Erlingsson, U., *et al.*: Extracting Training Data from Large Language Models. In: *30th USENIX Security Symposium (USENIX Security 21)*, pp. 2633–2650 (2021).
 - [69] Badhan Chandra Das, M. Hadi Amini, and Yanzhao Wu. 2025. Security and Privacy Challenges of Large Language Models: A Survey. *ACM Comput. Surv.* 57, 6, Article 152 (June 2025).
 - [70] Truong, J.-B., Maini, P., Walls, R.J., Papernot, N.: Data-Free Model Extraction. In: *Proceedings of the 2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 4769–4778. IEEE (2021).
 - [71] Hui, B., Yuan, H., Gong, N., Burlina, P., Cao, Y.: PLeak: Prompt Leaking Attacks Against Large Language Model Applications. In: *Proceedings of the 2024 ACM SIGSAC Conference*. ACM (2024).
 - [72] Du, H., Liu, S., Zheng, L., Cao, Y., Nakamura, A., Chen, L. (2025). Privacy in Fine-Tuning Large Language Models: Attacks, Defenses, and Future Directions. In: Wu, X., *et al.* *Advances in Knowledge Discovery and Data Mining. PAKDD 2025. Lecture Notes in Computer Science*, vol 15873. Springer, Singapore.
 - [73] Markov, T., Zhang, C., Agarwal, S., Eloundou, T., Less, T., Adler, S., Jiang, A., Weng, L.: A Holistic Approach to Undesired Content Detection in the Real World. *Proceedings of the AAAI Conference on Artificial Intelligence* 37, 15009–15018 (2023).
 - [74] Boreiko, V., Panfilov, A., Voracek, V., Hein, M., Geiping, J.: An Interpretable N-gram Perplexity Threat Model for Large Language Model Jailbreaks. In: *Proceedings of the 42nd International Conference on Machine Learning*, vol. 267 (2025).
 - [75] Barry S. Michaels *et al.* "Potential for Glove Risk Amplification via Direct Physical, Chemical, and Microbiological Contamination." *Journal of food protection* (2024): 100283.
 - [76] M. Zhu, S. Wei, L. Shen, Y. Fan, and B. Wu, "Enhancing Fine-Tuning based Backdoor Defense with Sharpness-Aware Minimization," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2023.
 - [77] Linyang Li, Demin Song, Xiaonan Li, Jiehang Zeng, Ruotian Ma, and Xipeng Qiu. 2021. Backdoor Attacks on Pre-trained Models by Layerwise Weight Poisoning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 3023–3032, Online and Punta Cana, Dominican Republic. Association for Computational Linguistics.
 - [78] Pearce, H., Ahmad, B., Tan, B., *et al.*: Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions. *Communications of the ACM* 68(2), 96–105 (2025).
 - [79] Kirkpatrick, J., Pascanu, R., Rabinowitz, N., *et al.*: Overcoming Catastrophic Forgetting in Neural Networks. *Proceedings of the National Academy of Sciences* 114(13), 3521–3526 (2017).
 - [80] Schick, T., Dwivedi-Yu, J., Dessi, R., *et al.*: Toolformer: Language Models Can Teach Themselves to Use Tools. In: *Advances in Neural Information Processing Systems*, vol. 36, pp. 68539–68551 (2023).
 - [81] Greshake, K., Abdelnabi, S., Mishra, S., *et al.*: Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In: *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, pp. 79–90. ACM (2023).
 - [82] Wachter, S., Mittelstadt, B., Floridi, L.: Why a Right to Explanation of Automated Decision-Making Does Not Exist in the General Data Protection Regulation. *International Data Privacy Law* 7(2), 76–99 (2017).
 - [83] JWang, P., Liu, X., Xiao, C.: CVE-Bench: Benchmarking LLM-based Software Engineering Agent's Ability to Repair Real-World CVE Vulnerabilities. In: *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies*, (2025).
 - [84] Wang, Y., Yang, C., Lan, S., *et al.*: End-Edge-Cloud Collaborative Computing for Deep Learning: A Comprehensive Survey. *IEEE Communications Surveys & Tutorials* 26(4), 2647–2683 (2024).